Phase 8 | Mock Interviews and Interview Simulation

Progress from project pitch to pressure-testing assumptions.

FastAPI Edition | Repository-grounded study material | Snapshot: 2026-10-24

Interviewer-led 30-minute mock

0-3 min: Introduction “Walk me through MetroInsights.” Follow-ups: who is the user? What is synthetic?

3-7 min: Request flow Trace `/api/predict` from JSON to response. Follow-up: why threadpool?

7-12 min: ML Explain 23 features, model choice, split, metrics. Follow-up: does 0.9946 R^2 mean real accuracy?

12-17 min: Mapping Explain canonical station name, aliases, line-expanded graph, interchange edge, Dijkstra. Follow-up: what if a station alias fails?

17-21 min: Scheduling Explain utilization and clearance. Follow-up: does it mean occupancy?

21-25 min: Reliability What happens when model file missing? What validation is incomplete?

25-30 min: Product judgment What would you build next and how would you prove benefit?

Ideal arc for 30 minutes

Be concise: disclose synthetic data early; walk one concrete request; explain one design choice deeply; state limitations without underselling the engineering; propose validation before deployment to operators.

45-minute extension

Add 10 minutes of source interrogation: feature assembly; historical aggregates and leakage; scheduler branch conditions; graph weight fallback; unknown station case. Add 5 minutes on testing and deployment. Be ready to identify exact files and outputs.

60-minute extension

Add design whiteboard: replace synthetic data with observed counts; propose ingestion schema, provenance, point-in-time features, time-based validation, model release process, API contract, monitoring and rollback. Add an outage scenario: route map alias mismatch plus a missing joblib artifact. Explain containment and diagnosis.

Pressure questions and model answers

“Your R^2 is 0.9946. Why should I trust it?” “I would trust it only as evidence that the model approximates generated scenario labels on a future-date synthetic holdout. It does not establish real passenger accuracy. We need observed counts, independent temporal test periods, calibration/error slices and domain review.”

“Why are you calling this ML if labels are made by code?” “It is a machine-learning prototype over synthetic labels. It demonstrates feature construction, training, baseline comparison and inference. It should be described transparently; the next step is replace labels with audited observations and repeat evaluation.”

“Your metro graph uses straight-line coordinates. Is it wrong?” “It uses distance only as a proxy between adjacent stops, multiplied by a fixed average speed, with fallback when coordinates look unreliable. It is better topologically than

connecting every pair directly, but not timetable routing. Official segment times and transfer paths would improve it."

"A route station is missing. What happens?" "The current topology-to-canonical map can fail and skip that edge. I would run a mapping audit at startup/CI, use stable IDs, and test interchanges. I would not claim full network coverage until the audit passes."

"Why not make everything async?" "Async is useful for waiting on I/O. sklearn and Pandas work is synchronous; converting the function syntax does not make CPU work asynchronous. The current route offloads it to a threadpool. If profiling shows CPU saturation, batch, cache, process-worker or job architecture may be needed."

"Why not put everything in PostgreSQL?" "The current inputs are static and read-mostly; CSV/JSON keep a prototype transparent. Observed counts, concurrent edits, provenance and transactional needs would justify a relational store."

Interviewer scorecard (1-5)

- Product clarity: says intended planner use and prototype scope.
- Evidence: cites actual modules, counts and route behavior.
- ML judgment: states synthetic validation limitation correctly.
- Algorithms: explains Dijkstra precondition, state mapping and complexity.
- API: understands ASGI/threadpool/input validation.
- Operations: distinguishes boarding clearance from onboard occupancy.
- Ownership: separates implementation from assumptions and proposed work.
- Communication: answers directly before adding detail.

Self-practice rules

Record yourself. Keep first answer under 60 seconds, then expand only when prompted. For every claim, name a file/metric and caveat. If uncertain, say what you would inspect or test. Do not invent real DMRC data access, production usage, observed accuracy, deployment verification, or personal impact.

How to use this series

Read Phase 1 first, then Phase 2 while opening the source files. Use Phases 3, 5, 6, and 7 to deepen technical understanding; rehearse with Phases 4 and 8; revise with Phase 9. The editable Markdown beside each PDF is the source of truth for corrections.

Evidence and honesty rules

MetroInsights is a planning prototype. Its demand targets and holdout are synthetic. Model metrics describe agreement with generated scenario labels, not real passenger counts or operational DMRC accuracy. Network topology, coordinates, capacities, headways, service frequency, and interchange splits are project data or configurable assumptions unless separately verified. Say "estimated" and explain the assumptions. Do not describe this as a live passenger-count, train occupancy, or production DMRC system.

Verified snapshot used in this edition

The cleaned station table contains 243 station records. The topology graph's destination count depends on the selected source and excludes the source itself; 242 destinations can be returned when all other modeled stations are connected. Do not say 226 or 228 without checking the current data. The project defines 11 route keys, including branch keys. The synthetic training series uses 243 stations x 728 dates x 19 hourly slots (05:00-23:00), or 3,361,176 rows. A chronological holdout begins 2026-10-01 and has 438,615 rows. The recorded HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, and MAPE 5.18%; Ridge MAE is 53.67 and historical station/day/hour mean MAE is 60.25. These are synthetic-only evaluation results.

Current stack and constraints

FastAPI + Uvicorn, Starlette/Jinja2, server-rendered HTML with JavaScript/CSS, Pandas, NumPy, scikit-learn, joblib, CSV/JSON, and a custom adjacency graph with heapq Dijkstra. No relational database, live ridership feed, authentication, or distributed job queue is present. The generated multi-million-row CSV is a training input; runtime inference loads compact serialized model/metadata plus station and operations inputs, rather than reading every training row on each request. Runtime startup imports routes and eagerly loads artifacts/builds the graph, so missing files can prevent startup. The current FastAPI migration has passed static compilation and diff checks in prior work, but an actual runtime smoke test was not possible in the offline environment; verify locally before claiming deployment verification.